



Universidad Cenfotec

Primer Estudio de Caso

Tema:

Aplicación de las pilas en el análisis de expresiones aritméticas

SOFT-10

Estructuras de Datos

SCV2

Integrante:

Fabiola Román Barrantes

Profesor:

Romario Salas Cerdas

Fecha de entrega:

19 de Octubre 2025

Introducción

Las estructuras de datos son la base fundamental del desarrollo de software eficiente y organizado. Entre ellas, las pilas (*stacks*) ocupan un lugar destacado debido a su comportamiento LIFO (*Last In, First Out*), donde el último elemento en entrar es el primero en salir. Este principio permite controlar de manera ordenada la ejecución de tareas anidadas o reversibles.

Una de las aplicaciones más relevantes de las pilas es el análisis y evaluación de expresiones aritméticas, como aquellas que contienen operadores, operandos y paréntesis. En este proceso, la pila permite administrar correctamente el orden de las operaciones, respetando las prioridades de los operadores y garantizando que las expresiones se evalúen de manera lógica y coherente.

El presente estudio describe por qué la pila es la estructura de datos más adecuada para esta funcionalidad, explica los beneficios de seguridad, orden y eficiencia que aporta, y presenta ejemplos concretos de su implementación en software de sistema.

Justificación del uso de la pila en el análisis de expresiones aritméticas

En el análisis de expresiones aritméticas, el objetivo principal es convertir una expresión en notación infija (por ejemplo, $3 + (4 * 2) - 7$) a una notación postfija o polaca inversa ($3 4 2 * + 7 -$), que puede evaluarse de manera directa por una máquina. Este proceso se realiza mediante el conocido algoritmo de Dijkstra, conocido como *Shunting Yard*, el cual depende completamente del uso de una pila.

La pila resulta ideal porque permite almacenar temporalmente operadores y paréntesis mientras se analiza la expresión carácter por carácter. Cada vez que se encuentra un operador, la pila determina si debe apilarse o desapilar otros operadores con mayor o igual precedencia antes de continuar. Este comportamiento secuencial y jerárquico encaja perfectamente con la filosofía LIFO: el último operador ingresado es el primero en resolverse cuando se alcanza un cierre de paréntesis o una prioridad menor.

Por ejemplo, si analizamos la expresión $(3 + 4) * 2$, la pila gestiona internamente los paréntesis y los operadores así:

1. Se apila '(' al encontrarlo.
2. Se apila '+' al leerlo dentro de los paréntesis.
3. Al cerrar ')', se desapila '+' para aplicarlo a los operandos 3 y 4.
4. Finalmente, se apila '*', que se aplicará después, respetando la precedencia.

Este flujo de operaciones sería imposible de manejar con estructuras como listas o colas, ya que estas no respetan el principio LIFO necesario para resolver operaciones anidadas en orden inverso.

Por tanto, la pila es la estructura natural y óptima para procesar expresiones jerárquicas, donde el último elemento ingresado debe resolverse antes que los anteriores.

Seguridad, orden y eficiencia que aporta la pila

Seguridad:

Las pilas permiten controlar errores de forma segura, ya que verifican condiciones como intentar desapilar (pop) un elemento cuando la pila está vacía. En el contexto de expresiones aritméticas, esta validación evita errores de ejecución cuando una expresión está incompleta o contiene paréntesis desbalanceados. También garantiza que cada operador tenga la cantidad correcta de operandos antes de ser evaluado.

Orden:

El orden lógico de las operaciones matemáticas (primero multiplicaciones/divisiones, luego sumas/restas, y evaluación dentro de paréntesis) se mantiene gracias al manejo ordenado de la pila. Cada vez que un operador es apilado o desapilado, se asegura que las operaciones se realicen en el orden jerárquico correcto, sin necesidad de recorrer varias veces la expresión completa.

Eficiencia:

Desde el punto de vista computacional, las operaciones básicas de una pila (push, pop, peek) tienen una **complejidad temporal $O(1)$** , lo cual las hace extremadamente eficientes. Esto significa que el análisis de una expresión puede realizarse con un único recorrido lineal, reduciendo tiempos de procesamiento. Además, al ser implementada mediante una lista enlazada o arreglo dinámico, la pila requiere poca memoria adicional.

En conjunto, la pila ofrece un equilibrio ideal entre seguridad, orden y eficiencia, lo que la convierte en una herramienta fundamental en el análisis sintáctico de expresiones dentro de compiladores, intérpretes y calculadoras.

Ejemplo concreto en software de sistema

Un ejemplo claro del uso de pilas en software de sistema es el funcionamiento interno de compiladores e intérpretes, como el Java Virtual Machine (JVM) o los analizadores sintácticos (*parsers*) de lenguajes como C o Python.

Durante la compilación, el intérprete transforma una expresión en código ejecutable mediante un proceso de análisis léxico y sintáctico. En ese proceso, las pilas son utilizadas para manejar los operadores, operandos y paréntesis, garantizando que las operaciones se evalúen en el orden correcto.

Por ejemplo, la JVM mantiene una pila de operandos donde se apilan valores conforme se leen las instrucciones del bytecode. La instrucción `iadd` toma los dos valores superiores de la pila, los suma y devuelve el resultado nuevamente a la parte superior. Este modelo es idéntico al funcionamiento de una calculadora postfija, demostrando cómo una pila permite implementar de forma simple y segura una evaluación aritmética.

Representación gráfica de las operaciones de una pila

El siguiente esquema muestra cómo se comporta la pila durante la conversión infija a postfija:

Expresión: $(2 + 3) * 4$

Paso 1: Lee '(' → Push '('

Pila: (

Salida:

Paso 2: Lee 2 → Agrega a salida

Pila: (

Salida: 2

Paso 3: Lee '+' → Push '+'

Pila: (, +

Salida: 2

Paso 4: Lee 3 → Agrega a salida

Pila: (, +

Salida: 2 3

Paso 5: Lee ')' → Pop hasta '('

Pila: (vacía)

Salida: 2 3 +

Paso 6: Lee '*' → Push '*'

Pila: *

Salida: 2 3 +

Paso 7: Lee 4 → Agrega a salida

Pila: *

Salida: 2 3 + 4

Paso 8: Fin de expresión → Vaciar pila

Pila: (vacía)

Salida final: 2 3 + 4 *

Este diagrama representa claramente cómo los operadores se apilan y desapilan en orden LIFO, garantizando que la multiplicación se ejecute al final, una vez resuelta la suma dentro del paréntesis.

Implementación práctica en Java

Objetivo:

Desarrollar un programa en Java que analice expresiones aritméticas utilizando pilas, verificando el balanceo de paréntesis, la conversión de notación infija a postfija y la evaluación del resultado.

Código:

AnalizadorExpresiones

```
import java.util.Scanner;
```

```
public class AnalizadorExpresiones {

    public static boolean verificarBalance(String expresion) {
        Pila pila = new Pila();
        String[] tokens = tokenizar(expresion);

        for (String t : tokens) {
            if (t.equals("(") || t.equals("[") || t.equals("{")) {
                pila.push(t);
            } else if (t.equals(")") || t.equals("]") || t.equals("}")) {
                if (pila.estaVacia()) return false;
                String tope = pila.pop();
                if ((t.equals(")") && !tope.equals("(")) ||
                    (t.equals("]") && !tope.equals "[")) ||
```

```

        (t.equals("{}") && !tope.equals("{}"))) {
            return false;
        }
    }
}
return pila.estaVacia();
}

```

```

public static String infijaAPostfija(String expresion) {
    StringBuilder salida = new StringBuilder();
    Pila pila = new Pila();
    String[] tokens = tokenizar(expresion);

    for (String t : tokens) {
        if (esNumero(t) || esVariable(t)) {
            salida.append(t).append(" ");
        } else if (t.equals("(")) {
            pila.push(t);
        } else if (t.equals(")")) {
            while (!pila.estaVacia() && !pila.peek().equals("(")) {
                salida.append(pila.pop()).append(" ");
            }
            if (!pila.estaVacia()) pila.pop(); // eliminar '('
        } else if (esOperador(t)) {
            while (!pila.estaVacia() && prioridad(pila.peek()) >= prioridad(t)) {
                salida.append(pila.pop()).append(" ");
            }
            pila.push(t);
        }
    }
}

```

```

while (!pila.estaVacia()) {
    salida.append(pila.pop()).append(" ");
}

return salida.toString().trim();
}

```

```

// ===== Evaluación de expresión postfija
=====

```

```

public static int evaluarPostfija(String expresion) {
    String[] tokens = expresion.split("\\s+");
    int[] pila = new int[tokens.length];
    int tope = -1;

    for (String t : tokens) {
        if (esNumero(t)) {
            pila[++tope] = Integer.parseInt(t);
        } else if (esOperador(t)) {
            int b = pila[tope--];
            int a = pila[tope--];
            int resultado = 0;

            switch (t) {
                case "+": resultado = a + b; break;
                case "-": resultado = a - b; break;
                case "*": resultado = a * b; break;
                case "/": resultado = a / b; break;
            }
            pila[++tope] = resultado;
        }
    }
}

```

```

    }
    return pila[tope];
}

// ===== Funciones de apoyo =====

public static boolean esOperador(String t) {
    return t.equals("+") || t.equals("-") || t.equals("*") || t.equals("/");
}

public static boolean esNumero(String t) {
    return t.matches("\\d+");
}

public static boolean esVariable(String t) {
    return t.matches("[a-zA-Z]+");
}

public static int prioridad(String t) {
    switch (t) {
        case "+": case "-": return 1;
        case "*": case "/": return 2;
        default: return -1;
    }
}

// Separa la expresión en tokens (números, variables, operadores y paréntesis)
public static String[] tokenizar(String expresion) {
    expresion = expresion.replaceAll("\\s+", "");
    return expresion.split("(?<=[-+*/()])|(?=[-+*/()])");
}

```

```
// ===== MENÚ PRINCIPAL =====

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int opcion;
    String expresion;

    do {
        System.out.println("\n=== Analizador de Expresiones con Pilas ===");
        System.out.println("1. Verificar balance de paréntesis");
        System.out.println("2. Convertir expresión infija a postfija");
        System.out.println("3. Evaluar expresión aritmética");
        System.out.println("4. Salir");
        System.out.print("Seleccione una opción: ");
        opcion = sc.nextInt();
        sc.nextLine();

        switch (opcion) {
            case 1:
                System.out.print("Ingrese la expresión: ");
                expresion = sc.nextLine();
                boolean balance = verificarBalance(expresion);
                System.out.println("Paréntesis balanceados: " + (balance ? "Sí" :
"No"));
                break;

            case 2:
                System.out.print("Ingrese la expresión: ");
                expresion = sc.nextLine();
                System.out.println("Expresión postfija: " + infijaAPostfija(expresion));
                break;
        }
    }
}
```

case 3:

```
System.out.print("Ingrese la expresión (solo números y + - * /): ");
expresion = sc.nextLine();
String postfija = infijaAPostfija(expresion);
int resultado = evaluarPostfija(postfija);
System.out.println("Expresión postfija: " + postfija);
System.out.println("Resultado: " + resultado);
break;
```

case 4:

```
System.out.println("¡Gracias por usar el analizador!");
break;
```

default:

```
System.out.println("Opción no válida.");
break;
```

```
}
```

```
} while (opcion != 4);
```

```
sc.close();
```

```
}
```

```
}
```

Pila

```
public class Pila {
```

```
    private Nodo cima; // referencia al último elemento ingresado
```

```
    public Pila() {
```

```
        this.cima = null;
```

```
}
```

```

// Inserta un nuevo elemento (token) en la pila
public void push(String dato) {
    Nodo nuevo = new Nodo(dato);
    nuevo.siguiente = cima;
    cima = nuevo;
}

// Elimina y devuelve el elemento superior de la pila
public String pop() {
    if (estaVacia()) {
        throw new RuntimeException("Error: la pila está vacía.");
    }
    String valor = cima.dato;
    cima = cima.siguiente;
    return valor;
}

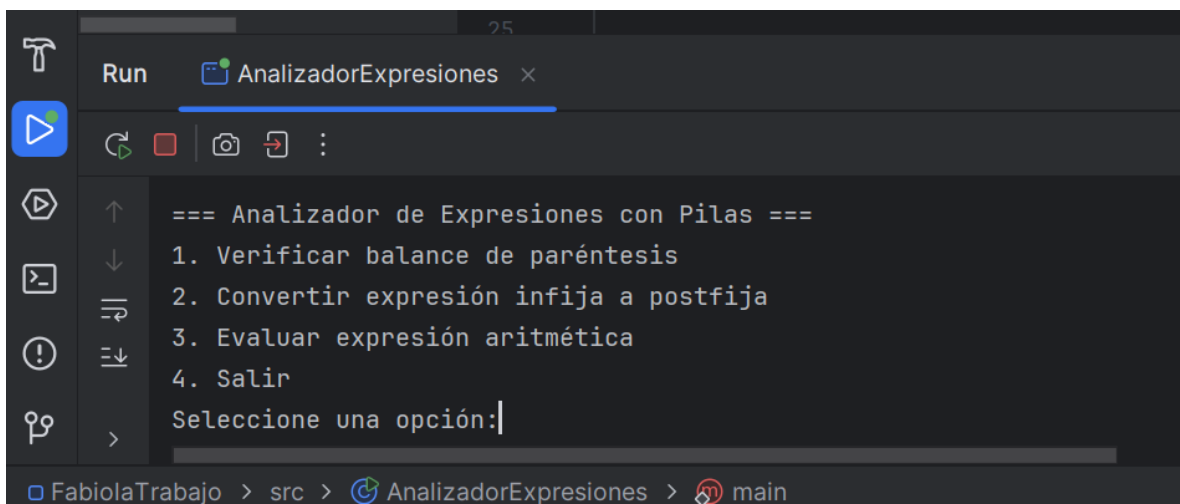
// Devuelve el elemento superior sin eliminarlo
public String peek() {
    if (estaVacia()) {
        throw new RuntimeException("Error: la pila está vacía.");
    }
    return cima.dato;
}

// Verifica si la pila está vacía
public boolean estaVacia() {
    return cima == null;
}
}

```

Nodo

```
public class Nodo {  
    String dato;  
    Nodo siguiente;  
  
    public Nodo(String dato) {  
        this.dato = dato;  
        this.siguiente = null;  
    }  
}
```



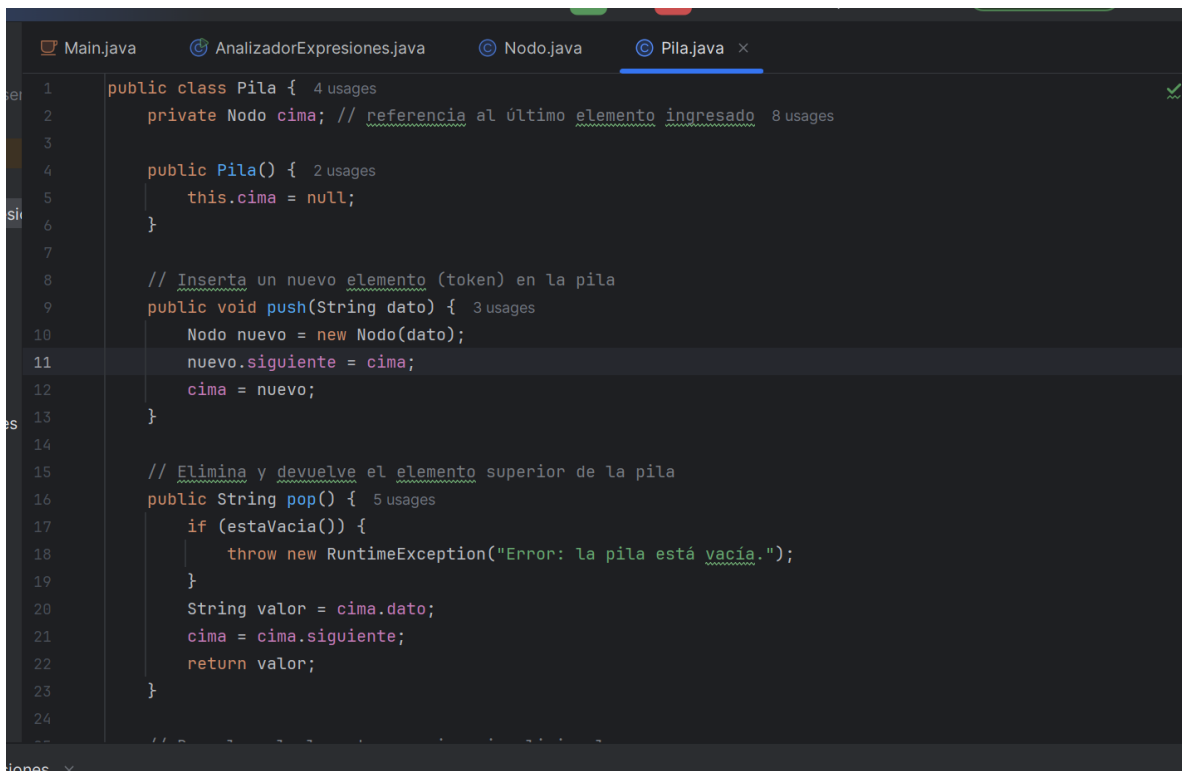
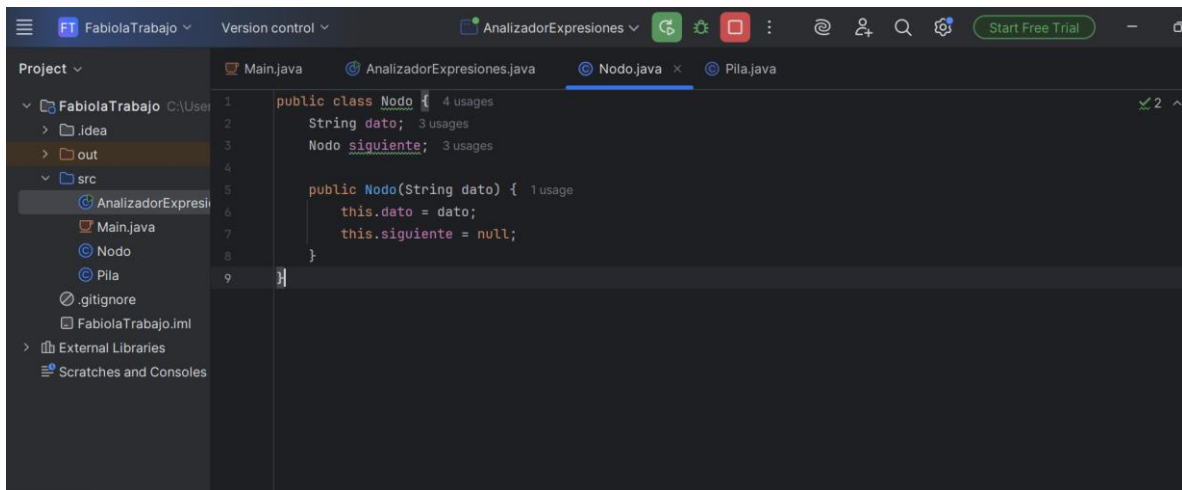
```
Run  AnalizadorExpresiones x  
25  
=== Analizador de Expresiones con Pilas ===  
1. Verificar balance de paréntesis  
2. Convertir expresión infija a postfija  
3. Evaluar expresión aritmética  
4. Salir  
Seleccione una opción:|  
FabiolaTrabajo > src > AnalizadorExpresiones > main
```

```
Run AnalizadorExpresiones x
2. Convertir expresión infix a postfija
3. Evaluar expresión aritmética
4. Salir
Seleccione una opción: 1
Ingrese la expresión: (3 + [5 * {2 + 1}])
Paréntesis balanceados: Si
```

```
Project v
v FabiolaTrabajo C:\User
> .idea
... > out
> src
AnalizadorExpresiones
Main.java
Nodo
Pila
.gitignore
FabiolaTrabajo.iml
> External Libraries
Scratches and Consoles

Main.java AnalizadorExpresiones.java x Nodo.java Pila.java
3 public class AnalizadorExpresiones {
6     public static boolean verificarBalance(String expresion) { 1 usage
9
10     for (String t : tokens) {
11         if (t.equals("(") || t.equals("[") || t.equals("{")) {
12             pila.push(t);
13         } else if (t.equals(")") || t.equals("]") || t.equals("}")) {
14             if (pila.estaVacia()) return false;
15             String tope = pila.pop();
16             if ((t.equals("(") && !tope.equals("(")) ||
17                 (t.equals("]") && !tope.equals("[")) ||
18                 (t.equals("}") && !tope.equals("{"))) {
19                 return false;
20             }
21         }
22     }
23     return pila.estaVacia();
24 }
25

Run AnalizadorExpresiones x
2. Convertir expresión infix a postfija
3. Evaluar expresión aritmética
4. Salir
Seleccione una opción: 2
Ingrese la expresión: (3 + 4) * 2
Expresión postfija: 3 4 + 2 *
=== Analizador de Expresiones con Pila ===
```



The screenshot shows an IDE window with the file `AnalizadorExpresiones.java` open. The code implements a method `verificarBalance` that checks if a given expression is balanced using a stack. The stack is represented by a `Pila` object. The method tokenizes the input expression and then iterates through the tokens. For each token, it checks if it is an opening parenthesis, a closing parenthesis, or an operator. If it is an opening parenthesis, it is pushed onto the stack. If it is a closing parenthesis, it checks if the stack is empty or if the top element is the corresponding opening parenthesis. If not, it returns `false`. If it is an operator, it checks if the stack is empty or if the top element is a closing parenthesis. If not, it returns `false`. After processing all tokens, it checks if the stack is empty. If it is, it returns `true`; otherwise, it returns `false`.

```
import java.util.Scanner;

public class AnalizadorExpresiones {

    public static boolean verificarBalance(String expresion) { 1 usage
        Pila pila = new Pila();
        String[] tokens = tokenizar(expresion);

        for (String t : tokens) {
            if (t.equals("(") || t.equals("[") || t.equals("{")) {
                pila.push(t);
            } else if (t.equals(")") || t.equals("]") || t.equals("}")) {
                if (pila.estaVacia()) return false;
                String tope = pila.pop();
                if ((t.equals(")") && !tope.equals("(")) ||
                    (t.equals("]") && !tope.equals("[") ||
                    (t.equals("}") && !tope.equals("{")))) {
                    return false;
                }
            }
        }
        return pila.estaVacia();
    }
}
```

The screenshot shows the same IDE window, but now the file `AnalizadorExpresiones.java` is open, showing the implementation of the `infijaAPostfija` method. This method converts an infix expression to a postfix expression using a stack. It tokenizes the input expression and then iterates through the tokens. For each token, it checks if it is a number or a variable, an opening parenthesis, a closing parenthesis, or an operator. If it is a number or variable, it is appended to the output string. If it is an opening parenthesis, it is pushed onto the stack. If it is a closing parenthesis, it pops elements from the stack and appends them to the output string until the corresponding opening parenthesis is found. If it is an operator, it checks if the stack is empty or if the top element has a higher or equal precedence. If not, it is pushed onto the stack. After processing all tokens, it pops the remaining elements from the stack and appends them to the output string.

```
public class AnalizadorExpresiones {

    public static String infijaAPostfija(String expresion) { 2 usages
        StringBuilder salida = new StringBuilder();
        Pila pila = new Pila();
        String[] tokens = tokenizar(expresion);

        for (String t : tokens) {
            if (esNumero(t) || esVariable(t)) {
                salida.append(t).append(" ");
            } else if (t.equals("(")) {
                pila.push(t);
            } else if (t.equals(")") ) {
                while (!pila.estaVacia() && !pila.peek().equals("(")) {
                    salida.append(pila.pop()).append(" ");
                }
                if (!pila.estaVacia()) pila.pop(); // eliminar '('
            } else if (esOperador(t)) {
                while (!pila.estaVacia() && prioridad(pila.peek()) >= prioridad(t)) {
                    salida.append(pila.pop()).append(" ");
                }
                pila.push(t);
            }
        }
        while (!pila.estaVacia()) {
            salida.append(pila.pop()).append(" ");
        }
        return salida.toString().trim();
    }
}
```

Análisis de resultados

El programa demuestra que las pilas permiten analizar correctamente expresiones aritméticas con distintos niveles de anidación.

Gracias a su comportamiento LIFO, el código logra mantener el orden de precedencia de los operadores y detectar errores en caso de paréntesis no balanceados.

Conclusiones y recomendaciones

El uso de pilas en el análisis de expresiones aritméticas constituye un ejemplo clásico y fundamental de aplicación de estructuras de datos. Gracias a su principio LIFO, la pila permite manejar con gran precisión las jerarquías y anidaciones que se presentan en los cálculos matemáticos, asegurando un flujo lógico y controlado en la evaluación de expresiones.

Además, el uso de la pila mejora la seguridad (al controlar desbordamientos o errores de sintaxis), mantiene el orden jerárquico de las operaciones, y garantiza una alta eficiencia con operaciones constantes en tiempo de ejecución.

Por todo lo anterior, las pilas no solo son una herramienta teórica de estudio, sino un recurso esencial en sistemas reales como compiladores, intérpretes y entornos de ejecución modernos. Su comprensión e implementación correcta permiten desarrollar software más robusto, eficiente y confiable.